Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

Content

- What is Retrieval-Augmented Generation
- How it works
- Real-World Applications
- Necessity and Benefits
- Challenges and Limitations
- Practice code

Retrieval-Augmented Generation

- Retrieval-Augmented Generation (RAG) stands as a significant advancement in the field of artificial intelligence, specifically in enhancing the capabilities of language models. This technique synergistically combines the strengths of traditional language models with the ability to dynamically incorporate relevant external data, leading to improved accuracy and contextual understanding in AI-generated content.

RAG necessity

- Large pre-trained language models have been shown to store factual knowledge in their parameters, and achieve state-of-the-art results when fine-tuned on downstream NLP tasks. However, their ability to access and precisely manipulate knowledge is still limited, and hence on knowledge-intensive tasks, their performance lags behind task-specific architectures. Additionally, providing provenance for their decisions and updating their world knowledge remain open research problems. Pretrained models with a differentiable access mechanism to explicit non-parametric memory have so far been only investigated for extractive downstream tasks.
- RAG models where the parametric memory is a pre-trained seq2seq model and the non-parametric memory is a dense vector index of Wikipedia, accessed with a pre-trained neural retriever.

RAG necessity

- parametric memory : the model's internal storage of knowledge and information, which is encoded within the model's parameters (weights) learned during training.
- Non parametric memory: storing and retrieving knowledge externally rather than relying solely on the model's internal parameters.

- The RAG system **retrieves** relevant information or documents from a separate knowledge base.
- Question -> Find Relevant Info -> Combine Question + Info -> Generate Better Answer

RAG architecture

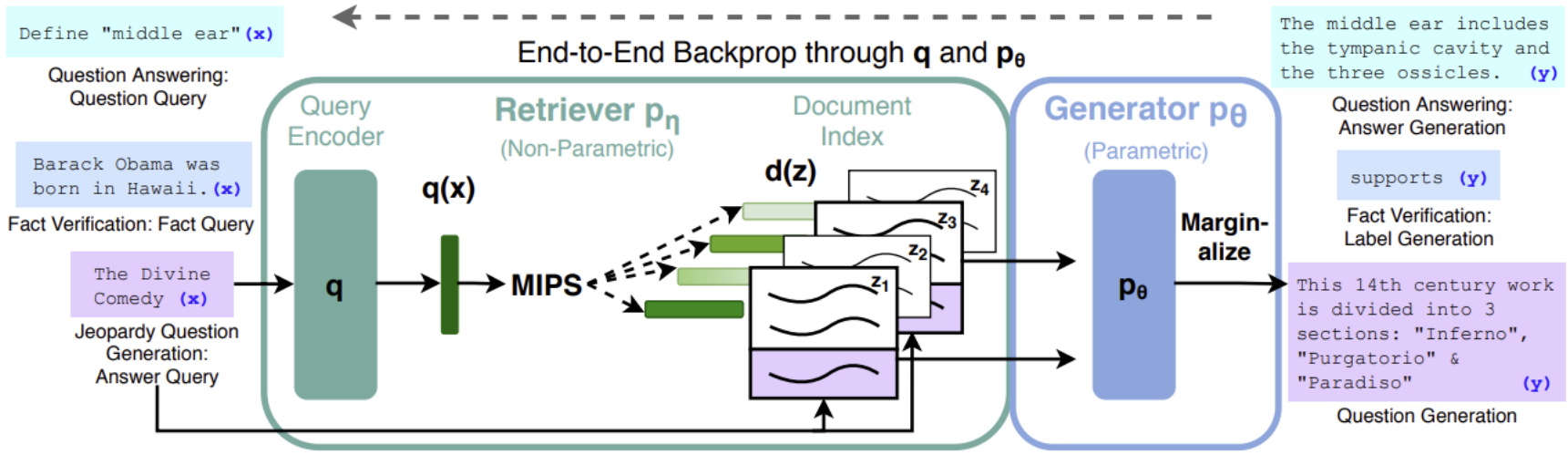

Figure 1: Overview of our approach. We combine a pre-trained retriever (*Query Encoder* + *Document Index*) with a pre-trained seq2seq model (*Generator*) and fine-tune end-to-end. For query x, we use Maximum Inner Product Search (MIPS) to find the top-K documents z_i. For final prediction y, we treat z as a latent variable and marginalize over seq2seq predictions given different documents.

RAG architecture

- It can overcome hallucination problem.
- By grounding the responses of LLMs in factual, up-to-date data retrieved from external knowledge sources, RAG offers a pathway to more reliable and versatile AI applications. The significance of RAG lies in its capacity to make LLMs more adaptable to real-world scenarios where access to timely and specific information is paramount, representing a fundamental shift towards more context-aware and dependable language generation.

How RAG works

- The typical RAG process unfolds through a series of carefully orchestrated stages, beginning with a user's query and culminating in an enhanced, information-backed response.
- The initial step involves the user providing a question or prompt, which the RAG system analyzes using Natural Language Processing (NLP) to discern the user's intent. Following this, the system initiates an information retrieval phase, searching its external knowledge base for content relevant to the query.
- This knowledge base is often structured as a vector database, containing numerical representations (embeddings) of a vast collection of documents.

How RAG works

- The retrieval process involves converting the user's query into an embedding and performing a similarity search to identify the most pertinent documents or data snippets.
- The retrieved information may then undergo a ranking and filtering process to ensure that the most relevant context is selected.
- Subsequently, the original query is augmented with this retrieved context, creating a richer, more informative prompt that is then fed into the large language model. The LLM processes this augmented prompt, drawing upon both its pre-trained knowledge and the newly provided context, to generate a coherent and contextually appropriate response.
- In some implementations, the generated response may undergo a final post-processing stage to further refine its quality through fact-checking, summarization, or formatting.

Basic building blocks

- These generally include a comprehensive collection of documents or a corpus that serves as the external knowledge base from which information will be retrieved. Another key component is the input from the user, which takes the form of a question or query that initiates the RAG process.
- It requires a mechanism to measure the similarity between the user's input and the documents within the corpus. This similarity measure is essential for enabling the system to identify and retrieve the most relevant information in response to the query.
- some introduce the concept of breaking down large documents into smaller, more manageable chunks, a process known as chunking, which is often employed to improve the efficiency and accuracy of the retrieval process.

Advantages of RAG

- The adoption of RAG yields several key benefits that address the limitations of traditional language models. Primarily, it enhances the accuracy of generated responses by grounding them in verifiable external sources, thereby significantly reducing the occurrence of factual errors and hallucinations.
- This grounding also allows the system to provide citations, increasing the trustworthiness of the information.
- Furthermore, RAG provides LLMs with access to the most current information, a crucial advantage in dynamic fields where knowledge evolves rapidly.
- This ability to incorporate real-time data overcomes the inherent issue of LLMs having static training datasets.
- In comparison to fine-tuning an LLM, RAG offers a more cost-effective and scalable method for integrating domain-specific or proprietary knowledge.

Advantages of RAG

- New information can be added to the external knowledge base without the need for computationally intensive model retraining.
- RAG also improves the transparency of the AI's reasoning by allowing users to see the sources of the information used in generating the response.
- Additionally, RAG can enhance the model's ability to handle ambiguous user queries by retrieving relevant context that clarifies the intent and enables a more targeted and accurate response.
- These advantages collectively make RAG a powerful technique for a wide array of applications, particularly those demanding high accuracy, access to real-time data, and efficient knowledge integration. However, it is important to note that the effectiveness of RAG can be influenced by the setup and maintenance of the retrieval component and the quality of the external knowledge base.

Advantages of RAG

- RAG can load data from diverse sources such as
- Load information from various file formats, including PDF documents , plain text files, and even content directly from web pages

Success of RAG

- Modern RAG frameworks and libraries play a crucial role in abstracting away much of the underlying complexity associated with implementing different retriever types, thereby making it easier for developers to experiment and select the most appropriate method for their specific RAG application. While dense retrieval methods have become increasingly favored for their semantic understanding capabilities, sparse retrieval and hybrid approaches continue to be relevant depending on the particular use case and data characteristics.

Success of RAG

- Effective prompt engineering is therefore of paramount importance for the success of RAG implementation. The manner in which the retrieved information is presented to the LLM has a significant impact on the quality and relevance of the generated response. Well-designed prompts can assist the LLM in focusing on the most pertinent details and synthesizing them into a comprehensive and accurate answer

Method

- As shown in Figure 1, RAG models leverage two components: (i) a retriever $p_\eta(z|x)$ with parameters η that returns (top-K truncated) distributions over text passages given a query x and (ii) a generator $p_\theta(y_i|x, z, y_{1:i-1})$ parametrized by θ that generates a current token based on a context of the previous $i - 1$ tokens $y_{1:i-1}$, the original input x and a retrieved passage z.

Method

- To train the retriever and generator end-to-end, RAG treat the retrieved document as a latent variable. RAG propose two models that marginalize over the latent documents in different ways to produce a distribution over generated text. In one approach, RAG-Sequence, the model uses the same document to predict each target token.
- The second approach, RAG-Token, can predict each target token based on a different document. In the following, we formally introduce both models and then describe the pη and pθ components, as well as the training and decoding procedure.

Models

RAG-Sequence Model The RAG-Sequence model uses the same retrieved document to generate the complete *sequence*. Technically, it treats the retrieved document as a single latent variable that is marginalized to get the seq2seq probability $p(y|x)$ via a top-K approximation. Concretely, the top K documents are retrieved using the retriever, and the generator produces the output sequence probability for each document, which are then marginalized,

$$p_{\text{RAG-Sequence}}(y|x) \approx \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) p_\theta(y|x,z) = \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) \prod_i^N p_\theta(y_i|x,z,y_{1:i-1})$$

Models

RAG-Token Model In the RAG-Token model we can draw a different latent document for each target *token* and marginalize accordingly. This allows the generator to choose content from several documents when producing an answer. Concretely, the top K documents are retrieved using the retriever, and then the generator produces a distribution for the next output token for each document, before marginalizing, and repeating the process with the following output token, Formally, we define:

$$p_{\text{RAG-Token}}(y|x) \approx \prod_{i}^{N} \sum_{z \in \text{top-}k(p(\cdot|x))} p_\eta(z|x) p_\theta(y_i|x, z, y_{1:i-1})$$

Retrieval Quality Issues

- Missing Information: If the answer to a user's query doesn't exist in the knowledge base, the RAG system might generate a misleading response or fail to acknowledge the absence of information.
- Suboptimal Retrieval and Ranking: Even if the correct information is present, the retrieval mechanism might fail to identify and rank it highly enough, leading to irrelevant or incomplete context. This can be due to limitations in keyword matching, semantic search gaps, or popularity bias.
- Query-Document Mismatch: The way a user phrases a question might not perfectly align with how information is stored in the knowledge base.

Context Limitations

- Token Limits: LLMs have constraints on the amount of text they can process at once. When many documents are retrieved, essential information might be truncated during the consolidation process.
- Difficulty in Extracting the Answer: The LLM might struggle to pinpoint the exact answer from a large or noisy retrieved context, even if the answer is present.
- Handling Multiple Documents: Effectively integrating information from several retrieved documents can be challenging, potentially leading to incoherence or repetition.

Limitations of RAG (Generation Challenges)

- Failure to Properly Incorporate Retrieved Information: The LLM might not effectively utilize the retrieved context, over-relying on its internal knowledge or ignoring crucial information.
- Hallucinations Despite Relevant Context: Even with relevant information provided, the LLM might still generate incorrect or fabricated details.
- Output Format Issues: The generated response might not match the desired format (e.g., tables, lists) or might lack proper attribution of sources.

Limitations of RAG (System Complexity and Cost)

- Increased Complexity: RAG systems involve integrating a retrieval mechanism with the LLM, adding architectural complexity.
- Computational Resources: Running a RAG system requires significant computational resources for both retrieval and generation, especially with large datasets.
- Infrastructure and Maintenance: Setting up and maintaining the retrieval system, including vector databases and indexing pipelines, adds overhead.

Limitations of RAG (Evaluation Challenges)

- Difficulty in Holistic Evaluation: Assessing the overall quality and relevance of RAG outputs is complex and can be subjective. Traditional LLM evaluation metrics might not fully capture the nuances of retrieval and generation integration.
- Lack of Standardized Metrics: There's no single, universally agreed-upon way to evaluate the effectiveness of a RAG system.

Limitations of RAG (ethical issue)

- Bias Propagation: If the retrieval sources contain biases, the LLM can inherit and amplify these biases in its responses.
- Misinformation and Trust: Poorly implemented RAG systems can still spread misinformation if the retrieved data is unreliable.
- Privacy and Security: Handling sensitive data in the knowledge base and during retrieval requires careful consideration of privacy regulations and security measures.
- Intellectual Property and Attribution: Ensuring proper citation and avoiding copyright infringement when using external data sources is crucial

Summary of RAG

- Reduces "hallucinations": The AI is less likely to make up facts because it's grounded in real information.
- Access to specific knowledge: It can answer questions about things the AI wasn't directly trained on.
- Keeps information up-to-date: You can update the knowledge base without retraining the entire AI.

Reference

- https://arxiv.org/pdf/2005.11401
- https://github.com/huggingface/transformers/blob/master/examples/rag/
- https://huggingface.co/rag/

Question

Thank you